IBM Maximo - webMethods Integration

a scenario

Laurent Martin

2026/04/07

Contents

1 Overview **2**
- 1.1 webMethods Calling Maximo REST APIs . . . 2
- 1.2 Maximo Triggering webMethods Workflows . . . 3
- 1.3 Use Cases . . . 3
- 1.4 Prerequisites for scenarios . . . 3
- 1.5 Repository Structure . . . 3

2 Pattern: webMethods Calling Maximo REST APIs **4**
- 2.1 Prerequisites . . . 4
- 2.2 Processing Maximo OpenAPI Specification . . . 4

3 Pattern: Maximo Triggering a webMethods Workflow **5**

4 Pattern: ServiceNow resuming a webMethods Workflow **6**

5 Micro Service Runtime (MSR) **10**

6 About **11**
- 6.1 License . . . 11
- 6.2 Contributing . . . 11

7 Documentation **12**
- 7.1 Support . . . 12

8 Annex: Demo environment ”self-managed” on macOS **13**

9 Annex: Creation of on-prem database **14**

10 Annex: Available Maximo Objects in IBM App Connect **15**

This repository provides tools, patterns and documentation for integrating IBM Maximo with enterprise applications using IBM webMethods Hybrid Integration Platform (IWHI).

Chapter 1

Overview

This integration enables bidirectional communication between Maximo and webMethods, supporting key integration patterns.

1.1 webMethods Calling Maximo REST APIs

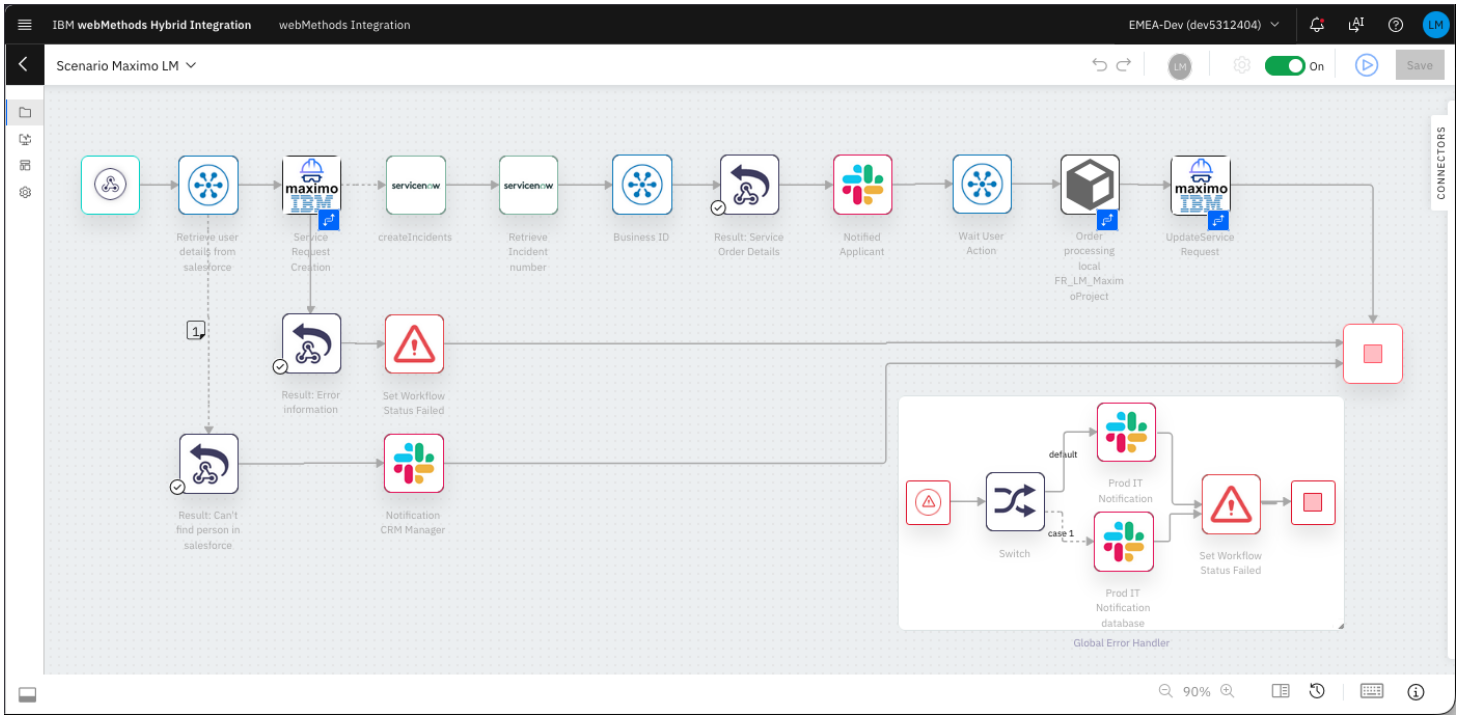

Figure 1.1: Overview of workflow

webMethods workflows can invoke Maximo REST APIs to:

- Query asset information
- Create or update work orders
- Retrieve service requests
- Perform CRUD operations on Maximo objects

The `/api` directory contains tools to process and optimize Maximo's OpenAPI specification for seamless import into webMethods.

1.2 Maximo Triggering webMethods Workflows

When objects are modified in Maximo (create, update, delete), Maximo can automatically trigger webMethods workflows to:

- Notify external systems of changes
- Orchestrate business processes across multiple applications
- Synchronize data with other enterprise systems
- Execute automated workflows based on Maximo events

1.3 Use Cases

- Asset Management: Sync asset data between Maximo and ERP systems
- Work Order Automation: Trigger workflows when work orders are created/updated
- Service Request Processing: Route service requests to appropriate systems
- Inventory Management: Update inventory across multiple platforms
- Compliance Reporting: Aggregate data from Maximo for reporting systems

1.4 Prerequisites for scenarios

- IBM Maximo Application Suite (MAS) instance
- IBM webMethods Hybrid Integration Platform access
- Other enterprise applications, such as SAP, Oracle ERP, ServiceNow, Salesforce, etc.

The Maximo instance main URL has the format (Documentation):

```
https://<sub_domain>.<mas_domain>/
```

1.5 Repository Structure

```
├── images/                 # Screen captures for documentation
├── logo/                   # Logo for Maximo connector
├── maximo/                 # OpenAPI specification processing tools
│   ├── filter-by-path.js   # Filter relevant API endpoints
│   ├── fix-oas.js          # Fix and enhance OpenAPI spec
│   ├── Makefile            # Build pipeline for API spec
│   ├── redocly.yaml        # Redocly configuration
│   └── api/                # OpenAPI specification
│       ├── oas.json        # Source
│       ├── oas.small.json  # filtered and fixed version
│       ├── oas.small.yaml  # yaml of above
│       └── oas.yaml        # yaml of source
├── src/                    # code
│   └── service_now_business_rule.js # servicenow
└── README.md               # This file
```

Chapter 2

Pattern: webMethods Calling Maximo REST APIs

2.1 Prerequisites

- IBM Maximo Application Suite (MAS) instance
- IBM webMethods Hybrid Integration Platform access
- Node.js (for API processing tools)
- Redocly CLI

2.2 Processing Maximo OpenAPI Specification

1. Get the OpenAPI specification for Maximo Manage from your Maximo instance:

   ```
   https://mas.manage.<mas_domain>/maximo/api/oas
   ```

> **Tip**
>
> API documentation can be consulted at: `https://mas.manage.<mas_domain>/maximo/oas3/api.html`

1. Save it in the `api` directory as `oas.json` (or use the one already saved)
2. Run the build process:

   ```
   make
   ```

   This will:

- Filter relevant API endpoints (`mxapisr` , `mxapiasset`)
- Bundle and optimize the specification
- Fix common issues (operationIds, enum types, server URLs)
- Validate the output
- Generate a clean OpenAPI spec ready for webMethods import

Import the processed OpenAPI specification into webMethods to:

- Auto-generate service connectors
- Build workflows that interact with Maximo
- Leverage Maximo's REST API capabilities

Chapter 3

Pattern: Maximo Triggering a webMethods Workflow

Configure Maximo to trigger webMethods workflows:

- Set up Maximo automation scripts or object structures
- Configure webhooks or message queues
- Implement event handlers in webMethods to process Maximo events

Chapter 4

Pattern: ServiceNow resuming a webMethods Workflow

- In webMethods Integration
- In your workflow, insert a utility connector: `Wait for Callback` in section `Suspend and Resume Workflow`

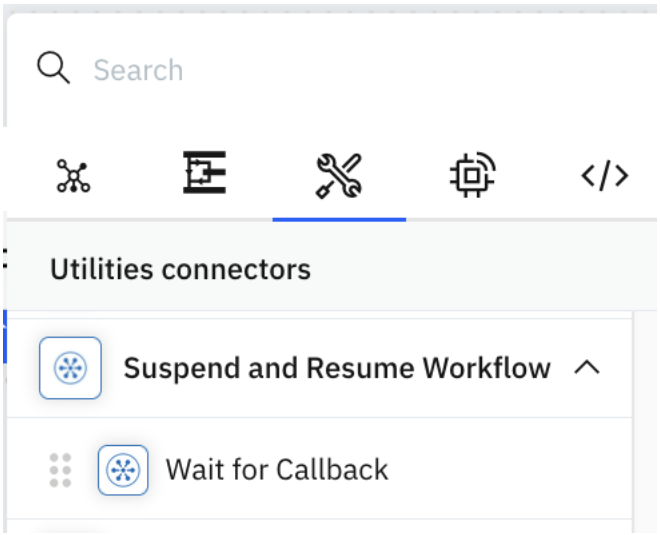

Figure 4.1: Resume connector in palette

In the configuration form: Copy the Webhook URL:

`https://<your-iwhi-instance>/runflow/resume/<step id>/(execution_id)`

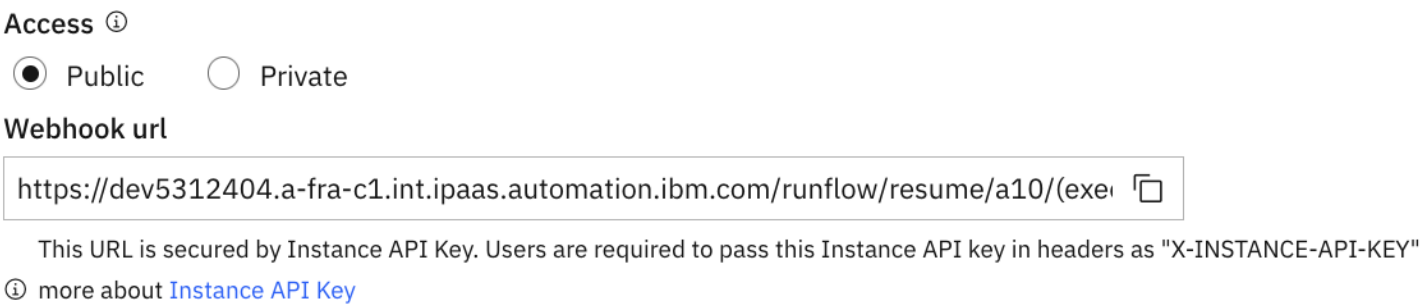

Figure 4.2: Resume connector configuration

- In ServiceNow
- Go to: `All > System Web Services > Outbound > REST Message`

- Create a REST Message (Endpoint):

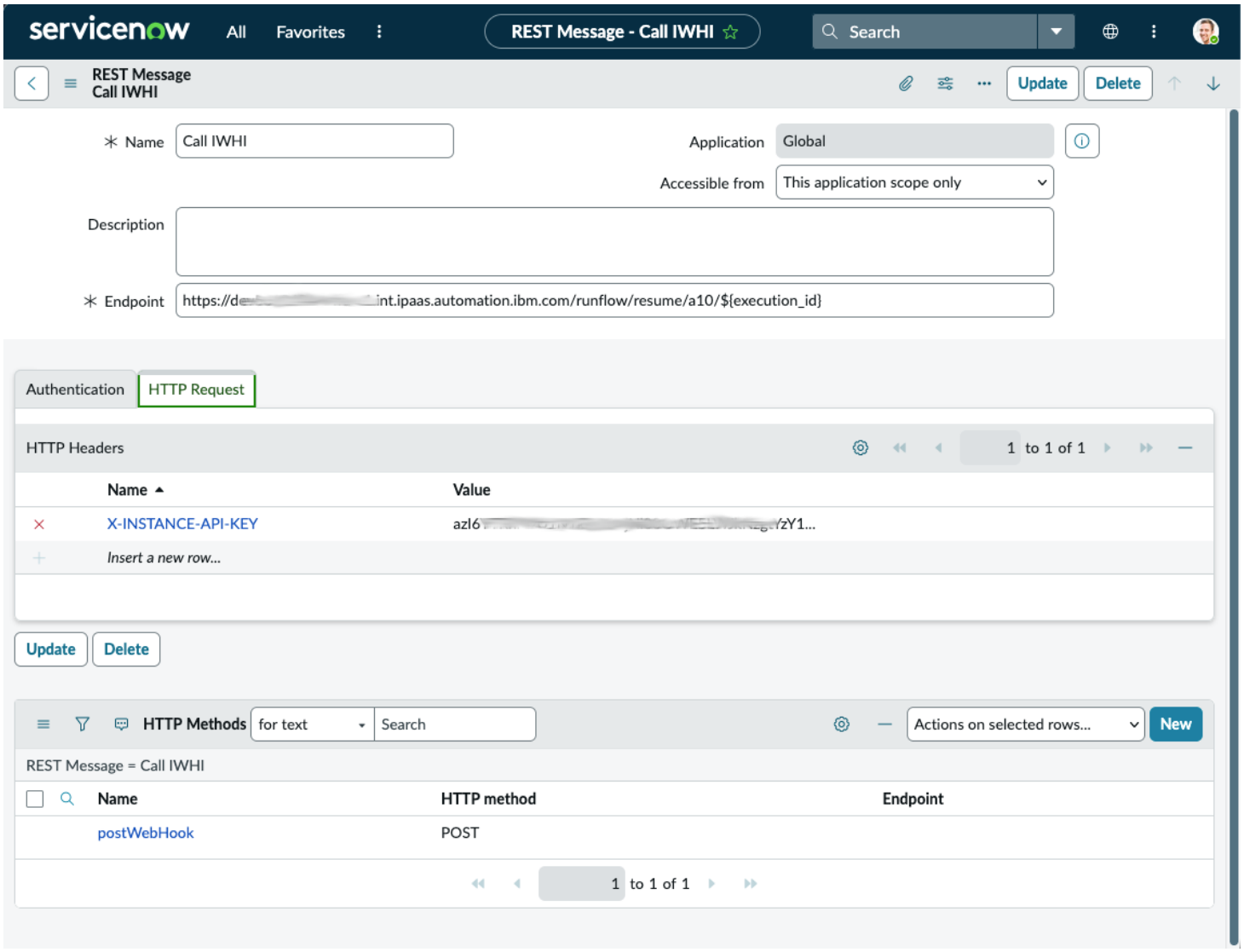

Figure 4.3: Create REST Endpoint

  - Name: `Call IWHI`
  - Endpoint: Paste as: `https://<your-iwhi-instance>/runflow/resume/<step id>/${execution_id}`
  - HTTP Request: add Header:
    * `X-INSTANCE-API-KEY` : `<your IWHI API key>`

- Create a HTTP Method (Message): in the same window:
  - Name: `postWebHook`
  - HTTP Method: `POST`
  - HTTP Request
    * Headers:
      · `Content-Type` : `application/json`
    * Content:

```
{
"event": "incident.updated",
"number": "${number}",
"short_description": "${short_description}",
"correlation_id": "${correlation_id}"
}
```

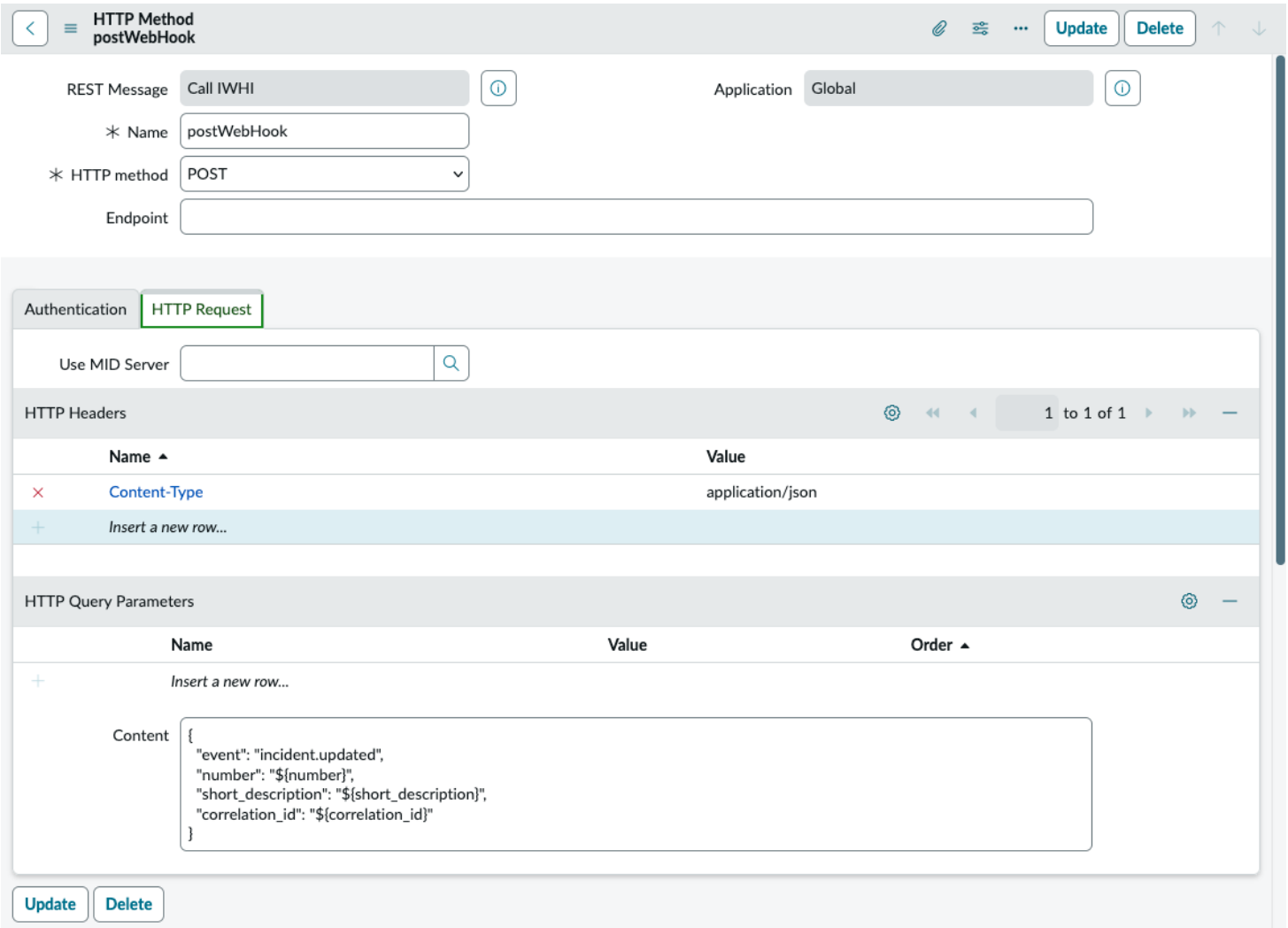

Figure 4.4: Create REST Message

Note

The variables: `execution_id` , `number` , `short_description` and `correlation_id` will be replaced with actual values in the script in the business rule.

- Go to: `All > System Definitions > Business Rules`
- Create a business Rule:

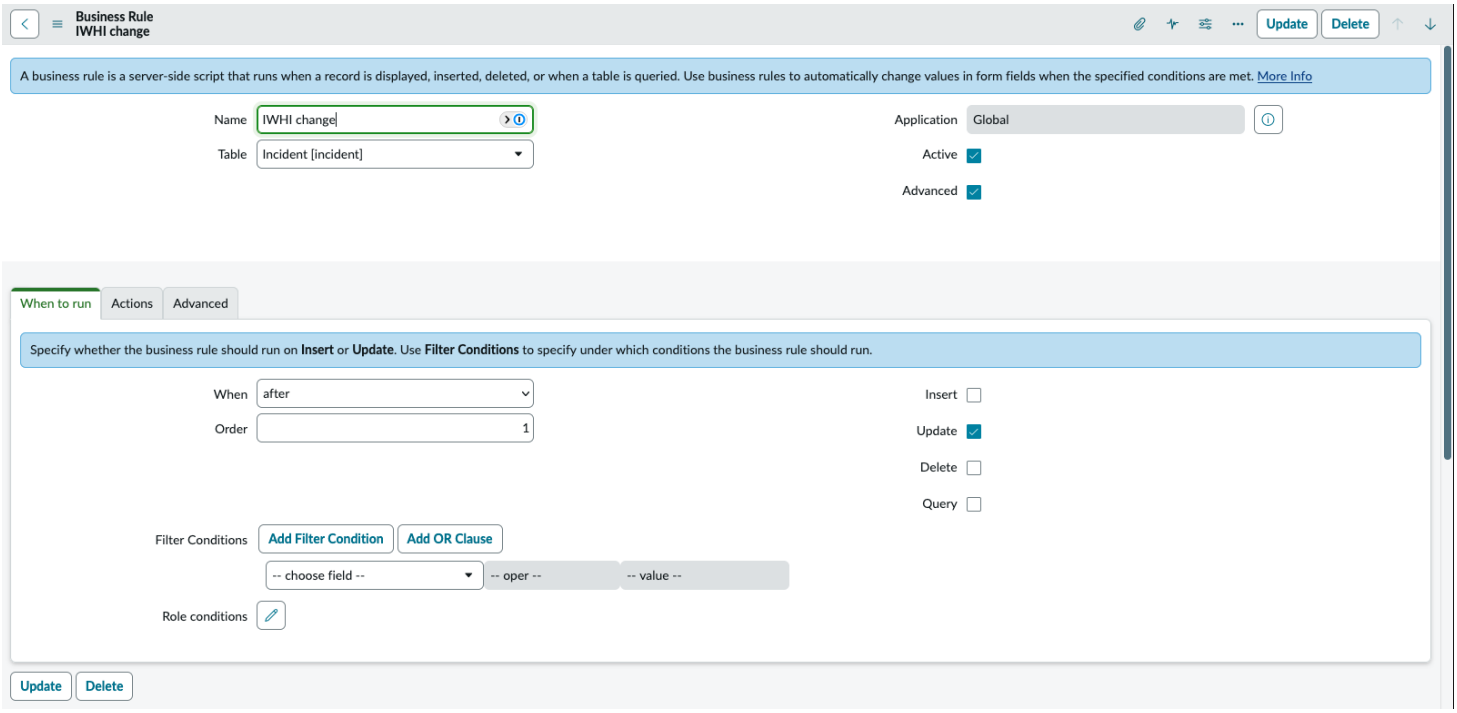

Figure 4.5: Create Business Rule

- Name: `IWHI Change`
- Table: `Incident`
- Active: `Yes`
- Advanced: `Yes`
- When to Run
  - When: `after`
  - Update: `Yes`
- Advanced:
  - Turn on ES12 mode: `Yes`
  - Script: javascript

Note

The field: `current.correlation_id` is populated at incident creation time by IWHI Integration.

Chapter 5

Micro Service Runtime (MSR)

The workflow simulates an on-prem service by using an Integration MSR accessing a database.

Chapter 6

About

6.1 License

This project is licensed under the Apache License 2.0 - see the LICENSE file for details.

6.2 Contributing

Contributions are welcome! Please feel free to submit issues or pull requests.

Chapter 7

Documentation

- Maximo REST API Documentation
- webMethods Integration Documentation

7.1 Support

For issues related to:

- Maximo: Consult IBM Maximo documentation or support
- webMethods: Consult IBM webMethods documentation or support
- This repository: Open an issue on GitHub

Chapter 8

Annex: Demo environment ”self-managed” on macOS

For the self-managed part, we deploy containers on macOS. The following setup is used:

```
brew install podman
```

```
podman machine init --cpus 4 --memory 4096 --disk-size 50
podman machine start
```

Chapter 9

Annex: Creation of on-prem database

Create a volume to have some persistency of the database:

```
podman volume create mysql-data
```

set database password in secrets.yml and set variable:

```
DB_PASSWORD=$(yq '.database.password' secrets.yaml)
```

The on-prem database runs in a container and is started like this:

```
podman run --detach --publish 3306:3306 --name=mysql --hostname=mysql-svc -e
↪  MYSQL_ROOT_PASSWORD=$DB_PASSWORD --volume mysql-data:/var/lib/mysql mysql:latest
```

> **Note**
>
> `mysql-svc` is a hostname visible by other containers, such as the Micro Service Runtime.

Create the database and table:

```
podman exec -i mysql mysql -u root -p$DB_PASSWORD < src/database_init.sql
```

Chapter 10

Annex: Available Maximo Objects in IBM App Connect

The following Maximo objects are available for integration in IBM App Connect:

```
IBM Maximo Asset Management is an enterprise asset management solution that enterprises can use for
 ↪  asset management, procurement and materials management, service management, work management, and
 ↪  contract management.
More info

Assets (mxapiasset)
Assets (mxasset)
Companies (mxapivendor)
Contracts (mxapicontract)
Crafts (mxapicraft)
Labors (mxapilabor)
Locations (mxapilocations)
Person groups (mxapipersongroup)
Person users (mxapiperuser)
Purchase orders (mxapipo)
Service addresses (mxapiseraddress)
Service requests (mxapisr)
Create service request
Retrieve service requests
Update service request
Update or create service request
Delete service request
Download service requests as CSV
Replace or create service request
Replace service request
Work orders (mxapiwo)
```

Fields in Maximo Create SR:

Field	Value
lean	1
Content-Type	application/json
properties	ticketuid,ticketid,reportedby,assetnum,reportedph
apikey	{{$project.params.apiKey}}
Accept	application/json
description_longdescription	Request : {{$request.body.description}} FROM webS
siteid	{{$request.body.SITEID}}
affectedperson	{{$request.body.affectedpersonid}}
description	{{$request.body.description}}
urgency	2
location	{{$request.body.location}}
status	{{$request.body.status}}

Field	Value
reportedby	{{$request.body.reportedby}}
assetnum	{{$request.body.assetnum}}
reportedemail	{{$a5.Contact[0].Email}}
reportedphone	{{$a5.Contact[0].Phone}}

Fields in Maximo Update SR:

Field	Value
lean	1
id	{{$transform.t3.stringifiedObject}}
Content-Type	application/json
Accept	application/json
apikey	{{$project.params.apiKey}}
x-method-override	PATCH
description_longdescription	Request : {{$request.body.description}} FROM webS

End of document